

Session 7: Error Handling, Debugging & OOP

Take-Home Practice Problems

Introduction

Welcome to Session 07 Practice!

These problems will help you master the key concepts from Session 07:

Topics Covered:

- Error handling: try/except/else/finally
- Raising and catching exceptions
- Input validation patterns
- Classes, objects, and methods
- Decorators: @property and @staticmethod
- Type annotations on functions, methods, and attributes

What Makes These Problems Better:

- ✓ **Clear requirements** - Each function/class has specific inputs and outputs
- ✓ **Built-in tests** - Run your code and see if it passes!
- ✓ **Formatted output** - All printing is handled for you
- ✓ **Focus on logic** - Just write the function/method bodies

How to Complete the Problems

1. Open the `SESSION_07_TAKE_HOME_PROBLEMS` folder
2. You'll find 4 Python files, one for each question
3. Each file has:
 - Class/function templates with clear documentation
 - TODO comments showing where to write your code
 - Test code that runs automatically
 - Expected output examples
4. Complete each function/method by replacing `pass` with your code
5. Run the file to test your implementation

6. Don't modify the test code at the bottom!

Question 1: Safe Calculator

File: question1_safe_calculator.py

Topics: Error handling, try/except, input validation

What You'll Do:

Implement functions with robust error handling:

1. `safe_divide(a, b)` - Division with error handling
2. `safe_convert(value, target_type)` - Type conversion with error handling
3. `get_valid_number(prompt)` - Input validation loop
4. `safe_list_access(lst, index)` - Safe list access with error handling

Key Skills:

- Using try/except for specific exceptions
- Handling `ZeroDivisionError`, `ValueError`, `TypeError`, `IndexError`
- Input validation with while loops
- Returning default values on error
- Type annotations on all function signatures

Important: All functions must include type annotations on parameters and return types (e.g., `def safe_divide(a: float, b: float) -> Optional[float]:`).

Expected Behavior:

```
1 # Safe division
2 safe_divide(10, 3)      # Returns 3.333...
3 safe_divide(10, 0)      # Returns None (no crash!)
4 safe_divide("10", 2)    # Returns None (type error!)
5
6 # Safe conversion
7 safe_convert("42", int)  # Returns 42
8 safe_convert("abc", int) # Returns None
9 safe_convert("3.14", float) # Returns 3.14
10
11 # Safe list access
12 safe_list_access([1,2,3], 1) # Returns 2
13 safe_list_access([1,2,3], 10) # Returns None
```

Hints:

- Use `except ZeroDivisionError` for division by zero
- Use `except (ValueError, TypeError)` to catch multiple types
- The input validation loop uses `while True + try/except`
- Return `None` as the default when an error occurs

Question 2: Student Class

File: question2_student_class.py

Topics: Classes, `__init__`, methods, `__str__`, error handling in classes

What You'll Do:

Implement a complete Student class:

1. `__init__(self, name, age)` - Initialize student with validation
2. `add_grade(self, grade)` - Add grade with validation (0-100)
3. `get_average(self)` - Calculate average grade
4. `get_letter_grade(self)` - Convert average to letter grade
5. `get_highest(self)` - Return highest grade
6. `get_lowest(self)` - Return lowest grade
7. `__str__(self)` - Friendly string representation

Key Skills:

- Defining classes with `__init__`
- Using `self` to store instance attributes
- Writing methods that operate on instance data
- Raising exceptions for invalid input
- Implementing `__str__` for readable output
- Type annotations on all methods and attributes

Important: All methods must include type annotations on parameters and return types. Instance attributes should also be annotated in `__init__`.

Expected Behavior:

```
1 alice = Student("Alice", 20)
2 alice.add_grade(85)
3 alice.add_grade(92)
4 alice.add_grade(78)
5
6 print(alice.get_average())      # 85.0
7 print(alice.get_letter_grade()) # "B"
8 print(alice.get_highest())     # 92
9 print(alice.get_lowest())      # 78
10 print(alice) # "Student(Alice, Age 20, Avg: 85.0, Grade: B)"
11
12 # Error handling
13 alice.add_grade(150) # Raises ValueError!
14 alice.add_grade(-10) # Raises ValueError!
```

Grade Scale:

- A: 90-100
- B: 80-89
- C: 70-79
- D: 60-69
- F: Below 60

Hints:

- Store grades in `self.grades = []`
- Use `raise ValueError()` for invalid grades
- Return 0 for average if no grades exist
- Use `max()` and `min()` for highest/lowest

Question 3: Bank Account

File: `question3.bank_account.py`

Topics: Classes, `@property`, `@staticmethod`, error handling

What You'll Do:

Implement a `BankAccount` class with decorators:

1. `__init__(self, owner, balance)` - Initialize with validation
2. `deposit(self, amount)` - Deposit with error handling
3. `withdraw(self, amount)` - Withdraw with error handling
4. `balance` - `@property` getter for current balance
5. `balance` setter - `@balance.setter` that validates the value (must be a number, cannot be negative)
6. `status` - `@property` returning "Good", "Low", or "Overdrawn"
7. `transaction_count` - `@property` for number of transactions
8. `is_valid_amount(amount)` - `@staticmethod` for validation
9. `__str__(self)` - Friendly display

Key Skills:

- Using `@property` for calculated attributes (getter)
- Using `@property.setter` for validated assignment
- Using `@staticmethod` for utility functions
- Raising exceptions for invalid operations
- Using `_` prefix for private attributes
- Transaction history tracking
- Type annotations on all methods and attributes

Important: All methods, properties, and attributes must include type annotations.

Expected Behavior:

```
1 acc = BankAccount("Alice", 500)
2 acc.deposit(100)      # Balance: 600
3 acc.withdraw(50)      # Balance: 550
4
5 print(acc.balance)    # 550
6 print(acc.status)     # "Good"
7 print(acc.transaction_count) # 2
8 print(acc)            # "Account(Alice, $550.00, Status: Good)"
9
10 # Error handling
11 acc.deposit(-50)      # Raises ValueError!
```

```

12 acc.withdraw(10000)  # Raises ValueError!
13
14 # Property setter with validation
15 acc.balance = 1000   # Works! Sets balance directly
16 acc.balance = -50    # Raises ValueError! Cannot be negative
17
18 # Static method
19 BankAccount.is_valid_amount(50)    # True
20 BankAccount.is_valid_amount(-10)   # False
21 BankAccount.is_valid_amount("abc") # False

```

Status Rules:

- “Good” - balance ≥ 100
- “Low” - balance ≥ 0 and ≤ 100
- “Overdrawn” - balance ≤ 0

Hints:

- Use `self._balance` (private convention) with `@property`
- For the setter, use `@balance.setter` and validate that the value is not negative
- Store transactions in `self._transactions = []`
- `is_valid_amount` checks type AND positive value
- Raise `ValueError` for invalid deposits/withdrawals

Question 4: Classroom Manager

File: `question4_classroom_manager.py`

Topics: Multiple classes, composition, file I/O, error handling

What You'll Do:

Implement a Classroom class that manages Student objects:

1. `__init__(self, name)` - Initialize classroom
2. `add_student(self, student)` - Add a Student object
3. `remove_student(self, name)` - Remove student by name
4. `get_student(self, name)` - Find student by name
5. `class_average` - @property for class-wide average
6. `top_student` - @property for highest-performing student
7. `get_honor_roll(self)` - Students with average ≥ 90
8. `save_to_file(self, filename)` - Save class data to CSV
9. `__str__(self)` - Class summary display

Key Skills:

- Composing classes (Classroom contains Student objects)
- Error handling for missing students, duplicate names
- @property for calculated class statistics
- File I/O to save structured data
- Type annotations on all methods and attributes
- Combining all Session 1-7 concepts

Important: All methods, properties, and attributes must include type annotations.

Expected Behavior:

```
1 classroom = Classroom("Python 101")
2
3 # Uses the Student class from Question 2
4 alice = Student("Alice", 20)
5 alice.add_grade(95); alice.add_grade(88); alice.add_grade(92)
6
7 bob = Student("Bob", 21)
8 bob.add_grade(75); bob.add_grade(82); bob.add_grade(78)
9
10 classroom.add_student(alice)
11 classroom.add_student(bob)
12
13 print(classroom.class_average)      # 85.0
14 print(classroom.top_student.name)  # "Alice"
```



```
15 print(classroom.get_honor_roll()) # [alice]
16 classroom.save_to_file("class.csv")
17 print(classroom)
18 # "Classroom(Python 101, 2 students, Avg: 85.0)"
```

CSV Format:

```
1 name,age,grades,average,letter_grade
2 Alice,20,95;88;92,91.7,A
3 Bob,21,75;82;78,78.3,C
```

Hints:

- Store students in `self.students = []` (list of Student objects)
- Use `raise ValueError` for duplicate student names
- For `class_average`, average all students' averages
- Use `sorted()` or `max()` with key for top student
- Join grades with semicolons: `";".join(map(str, grades))`

Bonus Challenge: Inventory System

If you finish all problems and want extra practice:

The Challenge:

Create a complete inventory management system with classes:

- Product class - name, price, quantity, category
- Inventory class - manages products, tracks sales
- Error handling for negative quantities, insufficient stock
- File persistence - save/load inventory from CSV
- Sales reporting with statistics

Example:

```
1 inventory = Inventory()
2 inventory.add_product(Product("Laptop", 999.99, 10))
3 inventory.add_product(Product("Mouse", 29.99, 50))
4
5 inventory.sell("Laptop", 2) # Sells 2 laptops
6 print(inventory.total_value) # Calculate total inventory value
7 inventory.save("inventory.csv")
```

Skills Used:

- Multiple classes with composition
- @property and @staticmethod decorators
- Comprehensive error handling
- File I/O for data persistence
- All concepts from Sessions 1-7!

Testing Your Solutions

Each file includes test code at the bottom. To test:

1. Complete the class/function implementations
2. Save the file
3. Run: `python question1.safe_calculator.py`
4. Check the output matches expected results
5. Verify all tests pass

✓ Success Criteria

You've mastered Session 07 when you can:

- Handle exceptions gracefully with try/except
- Create classes with `__init__`, methods, and `__str__`
- Use `@property` for calculated attributes (getters)
- Use `@property.setter` for validated assignment
- Use `@staticmethod` for utility functions
- Add type annotations to all functions, methods, and attributes
- Combine error handling with OOP
- Build systems with multiple cooperating classes

⚠ Common Mistakes to Avoid

- **Forgetting `self`:** Every method needs `self` as the first parameter!
- **Forgetting `self.`:** Use `self.name`, not just `name`
- **Bare `except`:** Always catch specific exceptions, not `except: pass`
- **Not raising errors:** Use `raise ValueError()` for invalid input
- **`@property` with `()`:** Don't use parentheses when calling properties!

Need Help?

- Review the session slides and examples
- Check the lecture notes for reference
- Test small parts of your code separately
- Print intermediate values to debug
- Ask questions in office hours!